Ingest Service — Test Assignment

Backend · NestJS + MongoDB · business requirements

1. Context

We receive patient events from external systems — EHRs, monitoring devices, partner integrations. The ingest service is the entry point for that traffic: it accepts events, processes them, and stores the result so that the rest of the platform can rely on it.

Your task is to build that service. Use NestJS (TypeScript) and MongoDB; everything else is your call.

2. What the service must do

Accept events over HTTP, process each one, and persist the outcome in a single MongoDB collection.

```
POST /events
{
  "patientId": "string",
  "type":      "string",
  "data":      { ... },
  "ts":        "ISO8601"
}
```

Processing a single event takes about **5 seconds** — it calls a slow external system. For this assignment, simulate that delay; the processing logic itself is irrelevant, only the timing is real.

3. Operating conditions

These are facts about the environment the service will live in, not a list of features. Decide yourself what they imply for your design.

- Traffic reaches **1000 events per minute** and stays there for hours. This is today's volume, not a ceiling — the business expects it to grow.
- Senders are third-party systems we do not control. They time out, they retry, and they can send us the same event more than once. Nothing guarantees that events reach us in the order they were created.
- For one patient, events describe a state that changes over time. The end result depends on the sequence in which that patient's events are applied.
- This is clinical data: an event may never be silently lost, and it may never be counted twice.
- The service runs 24/7 on infrastructure that restarts, deploys and kills processes at arbitrary moments, including in the middle of work.
- A sender must get an answer immediately; it will not wait for us to finish processing.

4. What we expect from you

- **Code you would be comfortable putting into production and handing over to a teammate.** We read the repository, not just the results.
- A working service that holds up under the conditions in section 3 — and a way for us to see that it does.
- A README with your architecture, the reasoning behind each significant decision, the trade-offs you accepted, and what you would do differently with more time.
- It runs locally in a couple of commands.
- Tests that cover the parts you consider risky.

Part of the exercise is working out which problems section 3 creates and how far to go with each one. We deliberately do not list them, and we do not expect every one of them to be solved — we expect you to have noticed them and to be able to explain your choices.

5. How we evaluate

What	What we look at
Code quality & structure — the heaviest criterion	Clean code with a modest feature set scores higher than a complete solution we struggle to read.
Problem awareness	Which risks in section 3 you spotted, and whether the design answers them.
Reasoning	Whether you can defend your decisions and name their downsides — in the README and in the follow-up call.
Not evaluated	UI, authentication, real clinical logic, CI/CD, infrastructure beyond running it locally.

Suggested effort: 4–8 hours. A smaller solution you can fully justify beats a large unfinished one. Deliver a Git repository with a readable commit history.